

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им.
Н.Э. Баумана

Факультет “Информатика и системы управления” Кафедра “Системы
обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:

Студент группы ИУ5-34Б

Мальков Н.А.

Преподаватель:

Гапанюк Ю.Е.

Москва 2025

Код main.py

```
class Section:
    def __init__(self, id, title, pages, doc_id):
        self.id = id
        self.title = title
        self.pages = pages
        self.doc_id = doc_id

class Document:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class SectionDocument:
    def __init__(self, doc_id, section_id):
        self.doc_id = doc_id
        self.section_id = section_id

documents = [
    Document(1, "Документ по безопасности"),
    Document(2, "Технический документ"),
    Document(3, "Руководство пользователя"),
    Document(4, "Архивный документ"),
]

sections = [
    Section(1, "Актуальные сведения", 5, 1),
    Section(2, "Общие сведения", 10, 1),
    Section(3, "Установка", 8, 2),
    Section(4, "Архив", 12, 2),
    Section(5, "Анализ данных", 6, 3),
    Section(6, "Приложение", 4, 4),
]

sections_documents = [
    SectionDocument(1, 1),
    SectionDocument(1, 2),
    SectionDocument(2, 3),
    SectionDocument(2, 4),
    SectionDocument(3, 5),
    SectionDocument(4, 6),
    SectionDocument(3, 1),
    SectionDocument(4, 2),
]

def get_sections_starting_with(letter, sections, documents):
    result = []
    for s in sections:
        if s.title.startswith(letter):
            for d in documents:
                if s.doc_id == d.id:
                    result.append((s.title, d.name))
```

```

    return result

def get_documents_with_min_pages(sections, documents):
    result = []
    for d in documents:
        pages_list = [s.pages for s in sections if s.doc_id == d.id]

        if pages_list:
            result.append((d.name, min(pages_list)))

    return sorted(result, key=lambda x: x[1])

def get_sections_and_documents_many_to_many(sections, documents, sections_documents):
    result = []
    for sd in sections_documents:
        for s in sections:
            for d in documents:
                if sd.section_id == s.id and sd.doc_id == d.id:
                    result.append((s.title, d.name))

    return sorted(result, key=lambda x: x[0])

if __name__ == "__main__":
    print("Задание B1:")
    res_b1 = get_sections_starting_with("A", sections, documents)
    for title, doc_name in res_b1:
        print(f"{title} - {doc_name}")

    print("\nЗадание B2:")
    res_b2 = get_documents_with_min_pages(sections, documents)
    for doc_name, min_p in res_b2:
        print(f"{doc_name} - {min_p}")

    print("\nЗадание B3:")
    res_b3 = get_sections_and_documents_many_to_many(sections, documents,
sections_documents)
    for title, doc_name in res_b3:
        print(f"{title} - {doc_name}")

```

Код test.py

```

import unittest
from main import (Section, Document, SectionDocument,
                  get_sections_starting_with,
                  get_documents_with_min_pages,
                  get_sections_and_documents_many_to_many)

class TestSectionDocumentLogic(unittest.TestCase):
    def setUp(self):
        self.documents = [
            Document(1, "Документ А"),
            Document(2, "Документ Б"),

```

```

        Document(3, "Документ Б")
    ]

    self.sections = [
        Section(1, "Анализ", 10, 1),
        Section(2, "Введение", 5, 1),
        Section(3, "Архитектура", 20, 2),
        Section(4, "База данных", 15, 3),
        Section(5, "Алгоритм", 8, 1)
    ]

    self.sections_documents = [
        SectionDocument(1, 1),
        SectionDocument(1, 2),
        SectionDocument(2, 3),
        SectionDocument(3, 4),
        SectionDocument(1, 5)
    ]

def test_sections_starting_with_A(self):
    result = get_sections_starting_with("А", self.sections, self.documents)
    expected = [
        ("Анализ", "Документ А"),
        ("Архитектура", "Документ Б"),
        ("Алгоритм", "Документ А")
    ]
    self.assertEqual(result, expected)

def test_min_pages_by_document(self):
    result = get_documents_with_min_pages(self.sections, self.documents)
    expected = [
        ("Документ А", 5),
        ("Документ Б", 15),
        ("Документ Б", 20)
    ]
    self.assertEqual(result, expected)

def test_many_to_many_relationship(self):
    result = get_sections_and_documents_many_to_many(
        self.sections, self.documents, self.sections_documents
    )

    expected = [
        ("Алгоритм", "Документ А"),
        ("Анализ", "Документ А"),
        ("Архитектура", "Документ Б"),
        ("База данных", "Документ Б"),
        ("Введение", "Документ А")
    ]
    self.assertEqual(result, expected)

if __name__ == "__main__":

```

```
unittest.main()
```

Результат выполнения программы

```
• → Malkov_Labs_3sem git:(rk2) ✖ /usr/local/bin/python3 /Users/nikitamalkov/Documents/Codes/Malkov_Labs_3sem/test.py
...
-----
Ran 3 tests in 0.000s
OK
```